COMSATS University Islamabad, Lahore campus

Assignment #2

Software Requirements Engineering

Submitted To:

Dr. Ghulam Rasool

Submitted by:

Samiullah Ashfaq
Sp24-BSE-107
Zain Yaqoob
Sp24-BSE-125
Muhammad Aqib Javed
Sp24-BSE-150
Section: A

Task 1: Requirements Analysis and Problem Identification

After carefully reviewing the requirements elicited in Assignment 1, we identified several issues. These problems fall under common requirement defects discussed in class: ambiguity, incompleteness, inconsistency, and non-verifiability. The table below lists each problematic requirement, describes the issue, and provides an updated corrected version.

#	Original Requirement	Problem Identified	Updated Requirement
1	User registration and confirmation of enrollment at the university.	Ambiguous — does not specify how enrollment is confirmed (document upload, university API, manual check?) or which universities are in scope.	The system shall allow students to register by uploading a valid university ID or enrollment letter. Where available, the system shall integrate with university student portals for automated verification.
2	Ride reservation and scheduling for car/bike pooling.	Incomplete — no mention of advance booking window, seat capacity limits per ride type, or what happens if a booking slot fills up before confirmation.	Students may book or schedule rides up to 24 hours in advance. Cars support a maximum of 4 passengers; bikes support 1. If seats fill during checkout, the system notifies the passenger and returns them to search results.
3	Payment system to allow multiple ways to pay.	Vague — supported payment methods are not listed, and refund and failed-payment behavior is undefined.	The system shall support JazzCash, EasyPaisa, debit/credit cards, and in-app wallet. Refunds for cancelled rides shall be processed within 48 hours. A failed payment prevents booking confirmation.
4	Negotiation for prices for pooled rides within a set limit.	Inconsistent — directly contradicts the business requirement of 'fixed shared pricing.' Allowing negotiation creates inconsistency and user disputes.	The system shall use fixed algorithm-based fare calculation for pooled rides based on distance, fuel cost, and passenger count. Peer-to-peer price negotiation is not permitted.
5	Rating and review for both drivers and passengers.	Incomplete — no defined minimum threshold, no stated consequence for consistently low ratings, and no mechanism to handle fake or malicious reviews.	Users with a rating below 2.5 / 5 after 10 or more reviews shall be flagged. Accounts may be suspended after three consecutive low-rated rides. Users may report suspicious reviews for admin review.
6	Safety features include an SOS button, live ride tracking, and emergency contact alerts.	Incomplete — no specification of who gets the alert, what data is shared, or expected response behavior when SOS is triggered.	When SOS is triggered, the system shall immediately share the user's live GPS location with pre-registered emergency contacts and the in-app safety team. A visible alert shall appear on the driver's screen.
7	Scalability: Ability to scale to support thousands of	Non-verifiable — 'thousands' is not a testable criterion. Non-functional requirements must	The system shall support a minimum of 5,000 concurrent active sessions with an average API response time

	concurrent clients.	be specific and measurable.	under 2 seconds and 99.9% monthly uptime.

Task 2: Data Flow Diagrams

Data Flow Diagrams show how information moves through the Student Ride-Sharing system. The Context Level DFD (Level 0) treats the system as a single process. The Level 1 DFD decomposes it into its core sub-processes and data stores.

2.1 Context Level DFD (Level 0)

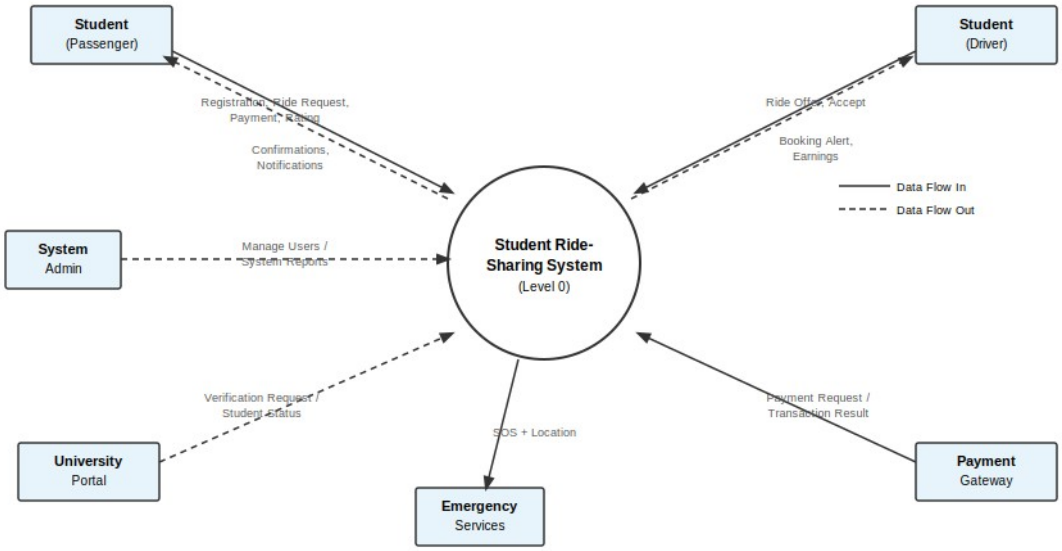

Figure 1: Context Level DFD — Student Ride-Sharing System

2.2 Level 1 DFD

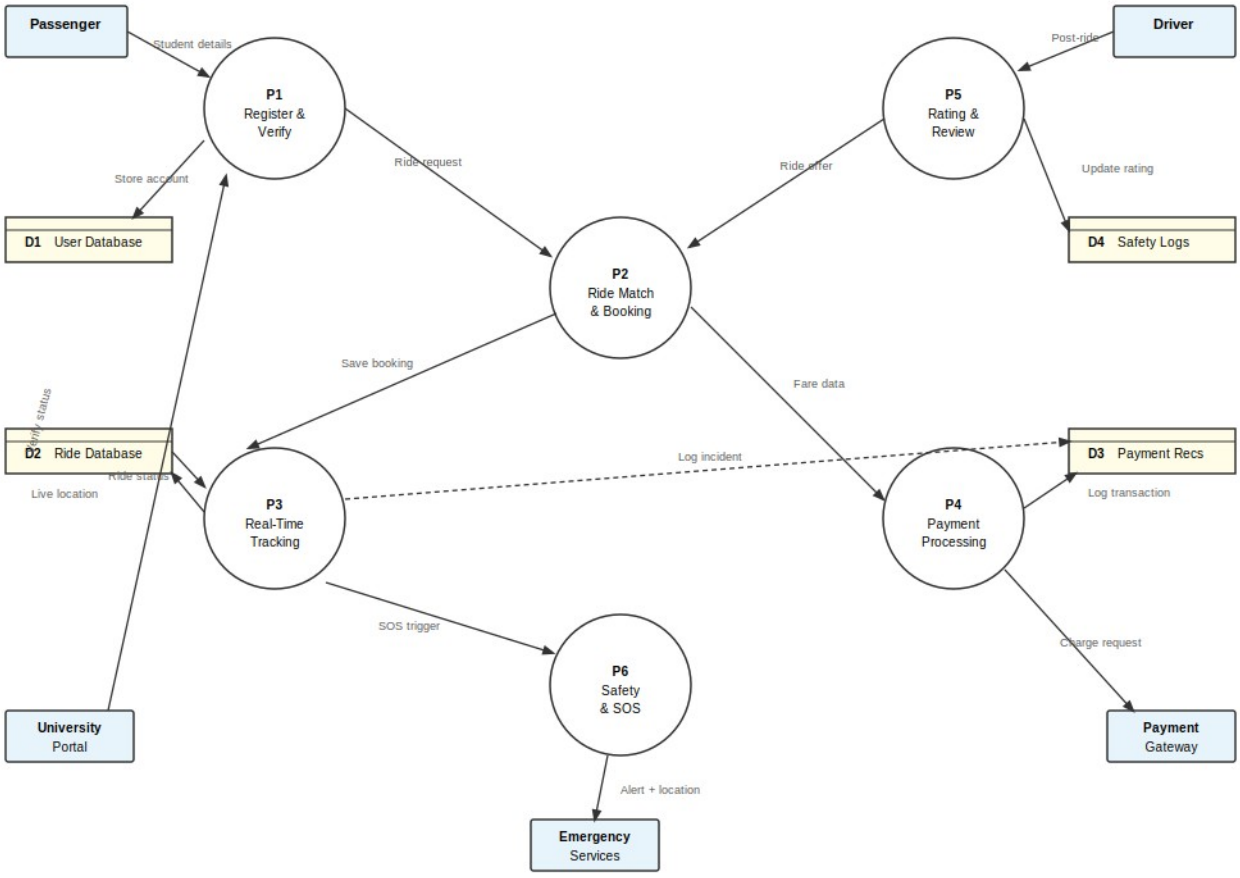

Figure 2: Level 1 DFD — Core Processes and Data Stores

Task 3: Actors and Use Cases

3.1 Actors

Actor	Description
Student (Passenger)	Verified university student who books rides, makes payments, and submits reviews.
Student (Driver)	Verified student who owns a vehicle and offers rides to fellow students.
System Administrator	Manages user accounts, handles disputes, monitors system performance and safety incidents.
University Portal	External system — provides student enrollment verification data.
Payment Gateway	External system — processes all financial transactions (JazzCash, EasyPaisa, cards).
Emergency Services	External entity — receives SOS alerts and live location data during safety incidents.

3.2 Use Cases

ID	Use Case	Primary Actor(s)
UC01	Register Account	Student (Passenger / Driver)
UC02	Verify Student Enrollment	System, University Portal
UC03	Log In / Log Out	Student (Passenger / Driver)
UC04	Offer a Ride	Student (Driver)
UC05	Search & Book a Ride	Student (Passenger)
UC06	Cancel a Ride	Student (Passenger / Driver)
UC07	Track Ride in Real Time	Student (Passenger)
UC08	Process Payment	Student (Passenger), Payment Gateway
UC09	Split Fare Among Passengers	System, Student (Passenger)
UC10	Rate & Review	Student (Passenger / Driver)
UC11	Trigger SOS Alert	Student (Passenger / Driver), Emergency Services
UC12	Manage User Accounts	System Administrator
UC13	View Ride History	Student (Passenger / Driver)
UC14	Earn & Redeem Loyalty Points	Student (Passenger / Driver)

Task 4: Use Case Model

The diagram below shows all actors, use cases, and their relationships including association lines, include, and extend relationships within the system boundary.

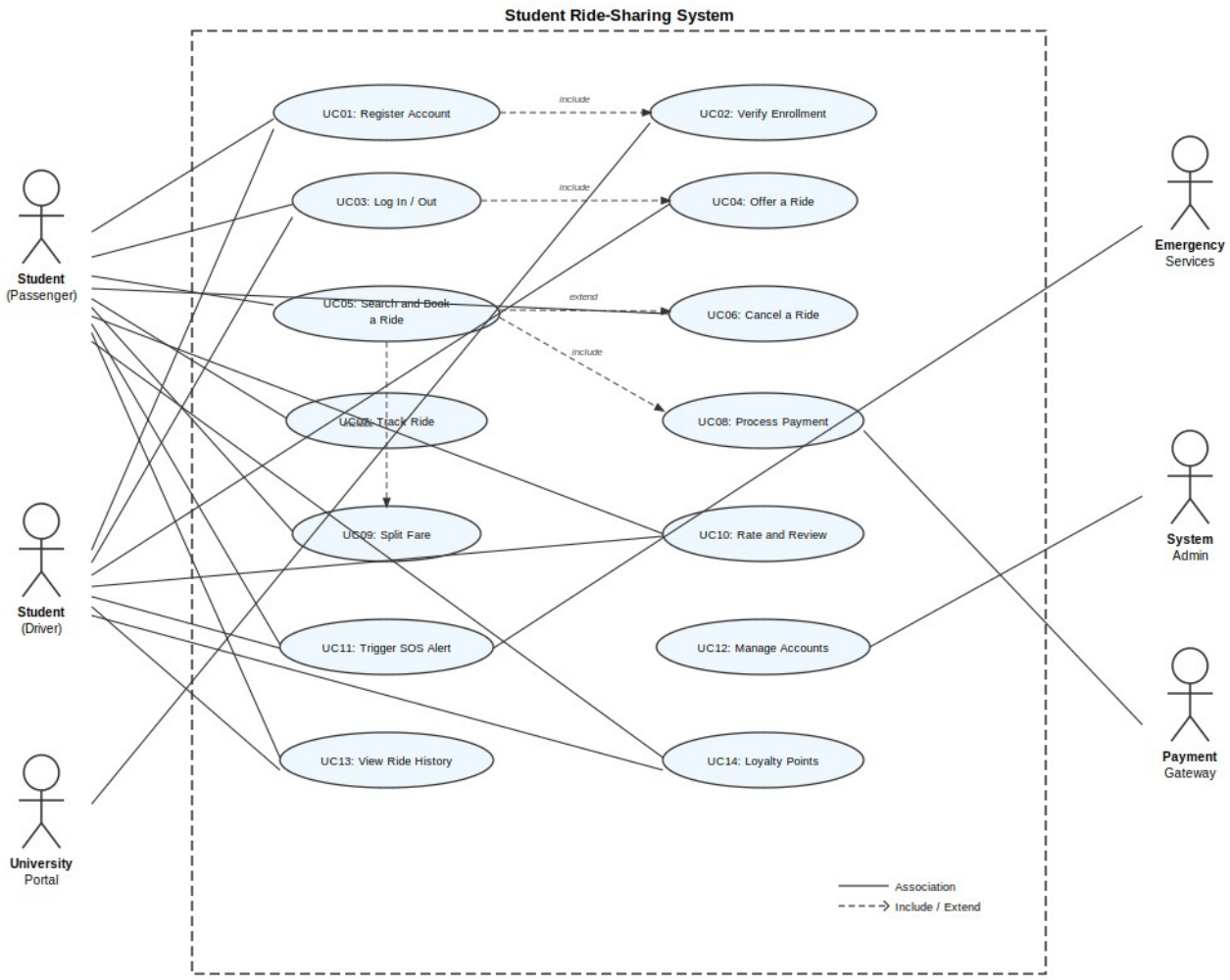

Figure 3: Use Case Diagram — Student Ride-Sharing Application

Task 5: Use Case Description — UC05: Search & Book a Ride

This use case is the most central operation in the system. It covers the full flow from a passenger searching for an available ride through to a confirmed booking.

Field	Details
Use Case ID	UC05
Use Case Name	Search & Book a Ride
Actor(s)	Student (Passenger) — Primary; System — Secondary
Description	A verified student searches for available rides to their destination and confirms a booking with a matching driver.
Preconditions	1. Student is registered and verified. 2. Student is logged in to the application. 3. At least one driver has an active ride offer on the requested route.
Postconditions	4. A confirmed booking record is stored in the Ride Database. 5. Both the passenger and driver receive a booking confirmation notification. 6. Payment is reserved and held until ride completion.
Main Flow	7. Passenger enters pickup location and destination in the search form. 8. System retrieves and displays matching available rides sorted by departure time and proximity. 9. Passenger reviews ride details: driver profile, star rating, fare, available seats, and vehicle type. 10. Passenger selects a ride and proceeds to the booking confirmation screen. 11. System calculates the fare; if multiple passengers, fare is split equally. 12. Passenger selects a payment method and confirms the booking. 13. System reserves the seat, charges the payment method, and sends confirmation to both parties.
Alternative Flows	**A1 — No rides available:** At step 2, if no matching rides exist, the system notifies the passenger. The passenger may modify the search parameters or schedule a future ride request. **A2 — Payment fails:** At step 6, if payment fails the system shows an error and prompts the passenger to choose another method. The booking is not confirmed until payment succeeds.
Exception Flows	**E1 — Session timeout:** If the session expires mid-booking, the system clears the pending state and

	redirects to the login screen. **E2 — Seat fills before confirmation:** If another passenger books the last seat between steps 4 and 7, the system notifies the current passenger and returns them to search results.
Includes	UC08: Process Payment, UC09: Split Fare Among Passengers
Extended By	UC06: Cancel a Ride
Priority	High — Core functionality of the application

Task 6: Decision Table and Decision Tree — Paying Fare

The following decision table covers all condition combinations and resulting actions for processing fare payment. The decision tree after it shows the same logic in a visual branching format.

6.1 Decision Table

Conditions	**R1**	**R2**	**R3**	**R4**	**R5**	**R6**	**R7**
Ride completed successfully?	Y	Y	Y	Y	N	N	N
Payment method valid?	Y	Y	Y	N	Y	N	Y
Multiple passengers (split fare)?	Y	N	Y	—	—	—	—
Cancellation within 15-min window?	—	—	—	—	Y	—	N
Actions							
Split fare equally among passengers		—		—	—	—	—
Charge full fare to single passenger	—		—	—	—	—	—
Send payment receipt to all parties				—	—	—	—
Credit earnings to driver wallet				—	—	—	—
Prompt update of payment method	—	—	—		—	—	—
Process full refund to passenger	—	—	—	—		—	—
Mark ride as incomplete, no charge	—	—	—	—	—		—

Apply cancellation penalty fee	—	—	—	—	—	—	

6.2 Decision Tree

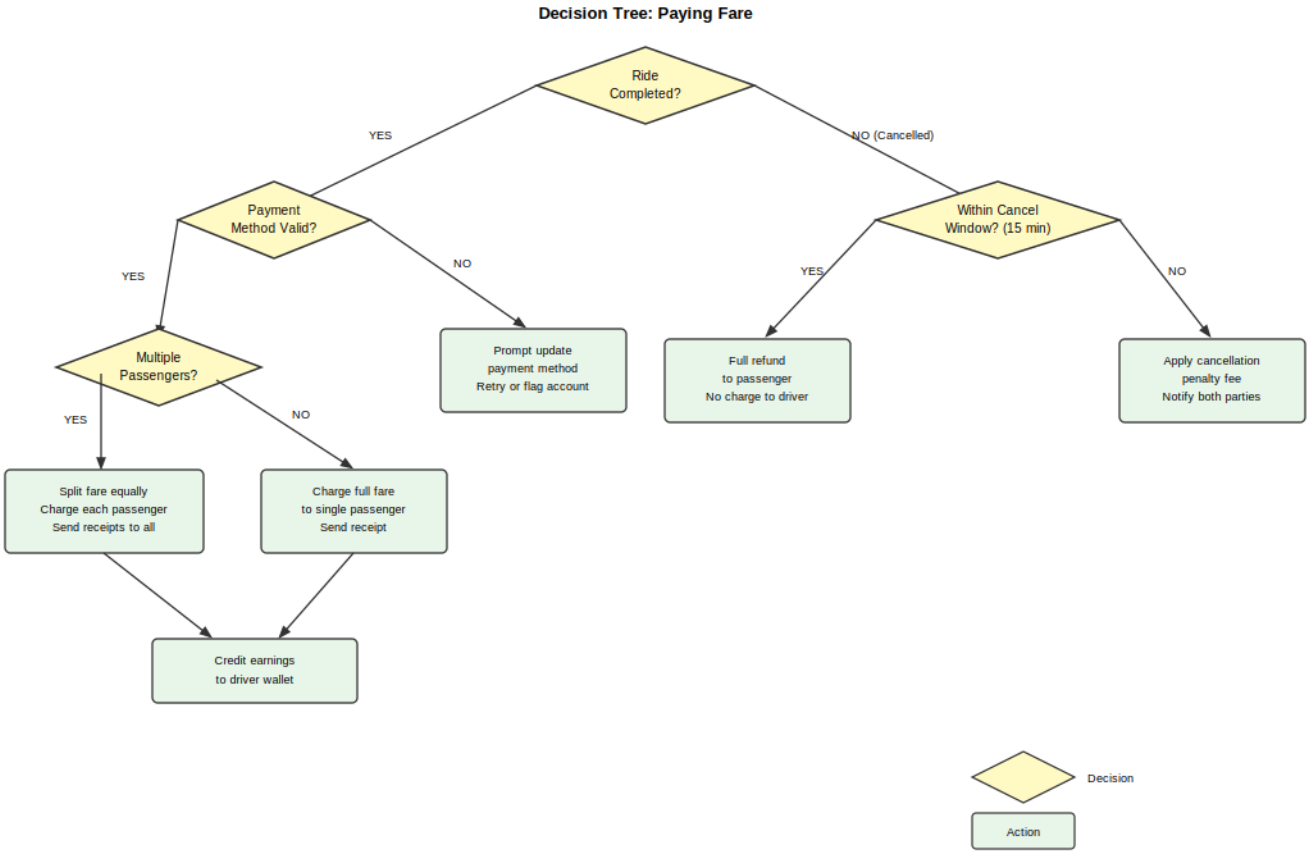

Figure 4: Decision Tree — Paying Fare